1.3 **Sequences**

Sequence

- A sequence is a list of objects arranged in a definite order
- Difference between set and sequence
  - A set has no order and no duplicated elements
  - A sequence has a specific order and elements may be duplicated
- Nomenclature: $a_1, a_2, \ldots a_n$

Sequence Examples

- 1, 2, 3, 2, 2, 3, 1 is a sequence, but not a set
- The sequences 1, 2, 3, 2, 2, 3, 1 and 2, 2, 1, 3 are made from elements of the set {1, 2, 3}
- The sequences 1, 2, 3, 2, 2, 3, 1 and 2, 1, 3, 2, 2, 3, 1 only switch two elements, but that alone makes them unequal

Types of Sequences

- Sequence may stop after n steps ***(finite)*** or go on for ever ***(infinite)***
- Formulas can be used to describe sequences
  - Recursive (التكراري)
  - Explicit (الصريح)

Recursive Sequences

- In a recursive sequence, the next item in the sequence is determined from previous values
- Difficult to determine say 100th element since previous 99 need to be determined first.
- Example:

  $a_1 = 1$

  $a_2 = 2$

  $a_n = a_{n-1} + a_{n-2}$

Explicit Sequences

- In an explicit sequence, the nth item is determined by a formula depending only on n
- Easier to determine any element
- Example: $a_n = 2 \times n$

Example 4. The recursive formula $c_1 = 5$, $c_n = 2c_{n-1}$, $2 \le n \le 6$, defines the finite sequence 5, 10, 20, 40, 80, 160. ◆

Example 5. The infinite sequence 3, 7, 11, 15, 19, 23, . . . can be defined by the recursive formula $d_1 = 3$, $d_n = d_{n-1} + 4$. ◆

Example 6. The explicit formula $s_n = (-4)^n$, $1 \le n < \infty$, describes the infinite sequence $-4, 16, -64, 256, \ldots$. ◆

Example 7. The finite sequence 87, 82, 77, 72, 67 can be defined by the explicit formula $t_n = 92 - 5n$, $1 \le n \le 5$. ◆

Example 8. An ordinary English word such as "sturdy" can be viewed as the finite sequence

$$\mathrm{s}, \mathrm{t}, \mathrm{u}, \mathrm{r}, \mathrm{d}, \mathrm{y}$$

composed of letters from the ordinary English alphabet. ◆

Strings

- Sequences can be made up of characters too
- Example: W, a, k, e, , u, p
- Removing the commas and you get a string: “Wake up”
- Strings best illustrate difference between sequences and sets
  - a, b, a, b, a, b, a, … is a sequence, i.e., “abababa…” is a string
  - The corresponding set is {a, b}

Linear Array

- Principles of sequences can be applied to computers, specifically, arrays. There are some differences though.
- Sequence
  - Well-defined
  - Modification of any element or its order creates new sequence
- Array
  - May not have all elements initialized
  - Modification of array by software may occur
  - Even if the array has variable length, we’re ultimately limited to finite length

Characteristic Functions

- A characteristic function is a function defining membership in a set
- $f_A(x) = \begin{cases} 1 \text{ if } x \in A \\ 0 \text{ if } x \notin A \end{cases}$
- Example, for the set A = {1, 4, 6}
  $f_A(1) = 1$, $f_A(2) = 0$, $f_A(3) = 0$, $f_A(4) = 1$, etc.

Programming Example

Characteristic functions may look unfamiliar, but consider the following code:

```
if (insert conditional statement here)
    return (1);
else
    return (0);
```

Example: $A = \{x \mid x > 4\}$

```
    if (x > 4) return (1);
    else return (0);
```

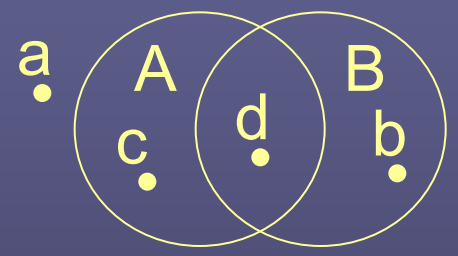
Properties of Characteristic Functions

Characteristic functions of subsets satisfy the following properties (proofs are on page 15 of textbook.)

- $f_{A\cap B} = f_A f_B$; that is $f_{A\cap B}(x) = f_A(x) f_B(x)$ for all x.
- $f_{A\cup B} = f_A + f_B - f_A f_B$;
  that is $f_{A\cup B}(x) = f_A(x) + f_B(x) - f_A(x) f_B(x)$ for all x.
- $f_{A\oplus B} = f_A + f_B - 2 f_A f_B$;
  that is $f_{A\oplus B}(x) = f_A(x) + f_B(x) - 2 f_A(x) f_B(x)$ for all x.

Proving Characteristic Function Properties

- Alternate way of doing proof is to enumerate all four cases and see how the result comes out
- Example: Prove $f_{A \cap B} = f_A f_B$



$f_A(a) = 0,\ f_B(a) = 0,\ f_A(a) \times f_B(a) = 0 \times 0 = 0 = f_{A \cap B}(a)$
$f_A(b) = 0,\ f_B(b) = 1,\ f_A(b) \times f_B(b) = 0 \times 1 = 0 = f_{A \cap B}(b)$
$f_A(c) = 1,\ f_B(c) = 0,\ f_A(c) \times f_B(c) = 1 \times 0 = 0 = f_{A \cap B}(c)$
$f_A(d) = 1,\ f_B(d) = 1,\ f_A(d) \times f_B(d) = 1 \times 1 = 1 = f_{A \cap B}(d)$

Representing Sets with a Computer

- Remember that sets have no order and no duplicated elements.
- The general need to assign each element in a set to a memory location gives it order in a computer.
- We can use the characteristic function to define a set using a computer.

Representing Sets with a Computer (continued)

- Assume *U* defines a finite universal set $U = \{x_1, x_2, x_3, \ldots, x_n\}$
- We can use characteristic function to represent subsets of *U*
- $f_A(x)$ is a sequence of 1's and 0's with the same number of elements as *U*
- $f_A(x) = 1$ is in position if corresponding element of *U*, x, is a member of *A*
- $f_A(x) = 0$ is in position if corresponding element of *U*, x, is not a member of *A*

Representing Sets with a Computer Example

- $U = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- $f_A(x) = 1, 0, 1, 0, 0, 1, 1, 0, 0, 1$
- $A = \{0, 2, 5, 6, 9\}$

More Properties of Sequences

- Any set with n elements can be arranged as a sequence of length n, but not vice versa. This is because sets have no order and duplicates are not allowed.
- Each subset can be identified with its characteristic function as a sequence of 1's and 0's.
- Characteristic function for universal set, U, is a sequence of all ones.

Countable and Uncountable

- A ***set*** is ***countable*** if it corresponds to some sequence.
  - Members of set can be arranged in a list
  - Elements have position
- All finite sets are countable
- Not all infinite sets are countable, and are therefore ***uncountable***
  - Best example is real numbers
  - E.g., what comes after 1.23534?

Strings and Regular Expressions

- Given a set *A*, the set *A** consists of all ***finite sequences*** of elements of *A*
- Example:
  - *A* = alphabet = {a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z}
  - *A** = words (the finite sequences, words, in A are not written with commas)
  - *A** contains ***all*** possible words, even those that are unpronounceable or make no sense such as “prsartkc”
- The ***empty sequence*** or ***empty string*** is represented with Λ

Catenation

- Two strings may be joined into a single string
- Assume $w_1 = s_1 s_2 s_3 s_4 \ldots s_n$ and $w_2 = t_1 t_2 t_3 t_4 \ldots t_k$
- The catenation of w_1 and w_2 is the sequence $s_1 s_2 s_3 s_4 \ldots s_n t_1 t_2 t_3 t_4 \ldots t_k$
- Notation: catenation of w_1 and w_2 is written as $w_1 \cdot w_2$ or $w_1 w_2$

Some Properties of Catenation

- If $w_1 \cdot w_2$ are elements of A*, then $w_1 \cdot w_2$ is an element of A*
- $w \cdot \Lambda = w$ and $\Lambda \cdot w = w$
- A subset *B* of *A** has its own set *B** which contains sentences made up from the words of *A*.
- For example:
  B = {John, Jane, swims, runs, well, quickly, slowly} is a subset of *A** where *A* = alphabet
  The element "Jane swims quickly" is an element of *B**.

Regular expressions

- The following is from http://etext.lib.virginia.edu/helpsheets/regex.html:

  "Regular expressions trace back to the work of an American mathematician by the name of Stephen Kleene (one of the most influential figures in the development of theoretical computer science) who developed regular expressions as a notation for describing what he called 'the algebra of regular sets.' His work eventually found its way into some early efforts with computational search algorithms, and from there to some of the earliest text-manipulation tools on the Unix platform (including ed and grep). In the context of computer searches, the '*' is formally known as a 'Kleene star.'"

Regular Expressions (continued)

- A regular expression on a set *A* is a recursive formula for a sequence.
- It is made up of the elements of *A* and the symbols (,), $\vee$, *,Λ
- The symbol Λ is a regular expression
- If x $\in$ *A*, the symbol x is a regular expression.
- If α and β are regular expressions, then the expression $\alpha\beta$ is regular.
- If α and β are regular expressions, then the expression $(\alpha \vee \beta)$ is regular.
- If α is a regular expression, then the expression $(\alpha)^*$ is regular.

Regular Expressions (continued)

A regular expression over A corresponds to a subset of A^*. This is called a ***regular subset*** of A^* or just ***regular set***. These subsets are built based on the rules presented on the next two slides. (You may want to use page 18 in the textbook as a reference in case I got one of these wrong.)

Rules of Regular Expressions

- The expression Λ corresponds to the set $\{\Lambda\}$, where Λ is the empty string in A^*.
- If $x \in A$, then the regular expression x corresponds to the set $\{x\}$
- If α and β are regular expressions corresponding to the subsets M and N of A^*, then $\alpha\beta$ corresponds to $M \cdot N = \{s \cdot t \mid s \in M \text{ and } t \in N\}$. Therefore, $M \cdot N$ is the set of all catenations of strings in M with strings in N.

Rules of Regular Expressions (continued)

- If the regular expressions α and β correspond to the subsets M and N of A^*, then $(\alpha \vee \beta)$ corresponds to $M \cup N$.
- If the regular expression α corresponds to the subset M of A^*, then $(\alpha)^*$ corresponds to the set M^*. Note that M is a set of strings from A. Elements from M^* are finite sequences of such strings, and thus may themselves be interpreted as strings from A. Note also that we always have $\Lambda \in M^*$.